Databases
SQL

Anna Monreale
Università di Pisa
anna.monreale@unipi.it

Introduction

- Standard language to define and query relational DB's, born in 1973
- **SQL (Structured Query Language)**
  - Interaction with DBMS
- **DDL (Data Definition Language)**
  - Creation of DB Schema
- **DCL (Data Control Language)**
  - Control of users authorization
- **DML (Data Manipulation Language)**
  - Instance manipulation

Queries

- **DML Instruction**
  - SELECT
  - One or more sub-queries correlated by operators on SETS

- **Sub-query**
  - Selection of some records
  - Projection (with aggregate functions)
  - Duplicates eliminations (DISTINCT)
  - Rename attributes
  - Order of the final results (ORDER BY)

Quering a DB

- **Core of the SELECT Instruction**
  - SELECT: projection, rename, distinct
  - FROM: cartesian product or join, alias
  - [WHERE]: selection
- **Other components**
  - [ORDER BY]

Quering a DB

```
SELECT attribute list
FROM tables list
[WHERE conditions]
```

– Select from the tables listend in `FROM` only the rows satisfying the conditions espressed in `WHERE` and extract only the gli attributes listed in `SELECT`

Table Students

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

Example Query

– List of name and surname of students of the bachelor degree

```
SELECT S.Surname, S.Name
FROM Students S
WHERE S.Program='bachelor'
```

Surname	Name
Rossi	Maria
Neri	Anna
Verdi	Fabio

FROM

- Lists the table or set of tables on which performing the query

Simple case: only one table

- `FROM Students`

or

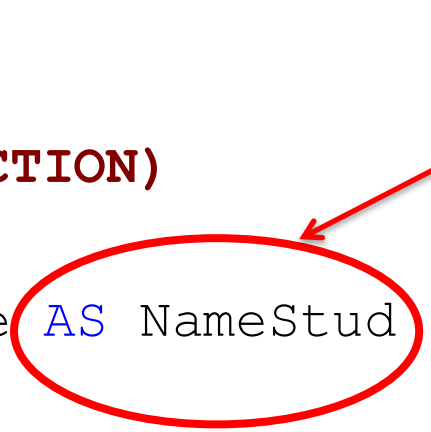
- `FROM Students S`

Projection: SELECT

- **Extract some columns from the table (Projection)**
- **SELECT [DISTINCT] <attributes> | ***
  - <attributes>
    - List of names of attributes
    - Use AS for renaming

Example

```
    SELECT * (WITHOUT PROJECTION)
OR
    SELECT S.Surname, S.name AS NameStud (PROJECTION)
```



Projection: SELECT

- **Schema of the result**
  - attributes of the original schema
- **Instance of the result**
  - Restriction of tuples to the specified attributes
- **Attention**
  - If the results does not contain keys the it may contain duplicates

Example of Projection

- List the surname degli students

```
SELECT Surname
FROM Students
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

Surname
Rossi
Neri
Verdi
Rossi
Bruni

Selection: `WHERE`

- **Selection of the tuples satisfying a specific conditions**
- **`WHERE <condition>`**
- **<condition>**
  - Condition of selection, Boolean connectors

Example

```
WHERE Surname='Rossi' AND Year>1
```

Example of Selection

- Extract information about students with surname Rossi

```
SELECT *
FROM Students
WHERE Surname='Rossi'
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
587614	Rossi	Luca	master	2	FT
276545	Rossi	Maria	bachelor	1	null

Example of Selection

- Extract surname of students with surname Rossi and Year > 1
-

```
SELECT *
FROM Students
WHERE Surname='Rossi' AND Year>1
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
587614	Rossi	Luca	master	2	FT

Complex Condition

- Extract information about students of the bachelor degree at the first or third year

```
SELECT *
FROM Students
WHERE Program = 'bachelor' AND (Year = 1 OR Year = 3)
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
200768	Verdi	Fabio	bachelor	3	FT

Condition“LIKE”

List people having a name starting with ‘M’ and containing ‘r' as third character

```
SELECT *
FROM Students
WHERE name LIKE ‘M_r%’ → M_r%e%’
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
937653	Bruni	Mario	master	1	CV

Manage NULL Values

– Students without a Supervisor

```
SELECT *
FROM Students
WHERE Supervisor is NULL
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null

DISTINCT

- The result may contain duplicates
- **Example**
  - List the “Program” in the table students

```
SELECT Program
FROM Students
```

Program
bachelor
bachelor
bachelor
master
master

```
SELECT DISTINCT Program
FROM Students
```

Program
bachelor
master

ORDER BY

- Sort tuples in the result

- **ORDER BY <attributes>**
  - **<attributes>**
    - List of attributes
    - <attribute> {ASC | DESC}

Example

```
ORDER BY Surname ASC, Name DESC
ORDER BY Surname DESC, Name DESC
ORDER BY Surname, Name
```

(default ASC)

- **Semantics**
  - Sorting tuples

Example: ORDER BY

- Extract information about students with surname Rossi

```
SELECT *
FROM Students
WHERE S.Surname='Rossi'
ORDER BY Surname, Name DESC
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
587614	Rossi	Luca	master	2	FT

FROM with JOIN

- In the relational data model data are split in different tables
- During the queries we need **to correlate data from different tables**
- It is possible to compute the **cartesian product**
- To express JOIN operation on the result of cartesian product, we apply the conditions in WHERE indicating the link between tables

Example of JOIN

STUDENTS

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

TEACHERS

Code	Surname	Name	Role	Department
FT	Monreale	Anna	Reseacher	Computer Science
CV	Tesconi	Maurizio	Reseacher	Engineering

StudentID	Surname	Name	Program	Year	Supervisor	Code	Surname	Name	Role	Departmentd
276545	Rossi	Maria	bachelor	1	null	FT	Monreale	Anna	Engineering	Computer Science
276545	Rossi	Maria	bachelor	1	null	CV	Tesconi	Maurizio	Reseacher	Engineering
485745	Neri	Anna	bachelor	2	null	FT	Monreale	Anna	Reseacher	Computer Science
485745	Neri	Anna	bachelor	2	null	CV	Tesconi	Maurizio	Reseacher	Engineering
200768	**Verdi**	**Fabio**	**bachelor**	**3**	**FT**	**FT**	**Monreale**	**Anna**	**Reseacher**	**Computer Science**
200768	Verdi	Fabio	bachelor	3	FT	CV	Tesconi	Maurizio	Reseacher	Engineering
587614	**Rossi**	**Luca**	**master**	**2**	**FT**	**FT**	**Monreale**	**Anna**	**Reseacher**	**Computer Science**
587614	Rossi	Luca	master	2	FT	CV	Tesconi	Maurizio	Reseacher	Engineering
937653	Bruni	Mario	master	1	CV	FT	Monreale	Anna	Reseacher	Computer Science
937653	**Bruni**	**Mario**	**master**	**1**	**CV**	**CV**	**Tesconi**	**Maurizio**	**Reseacher**	**Engineering**

Example of JOIN

STUDENTS

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

TEACHERS

Code	Surname	Name	Role	Department
FT	Monreale	Anna	Reseacher	Computer Science
CV	Tesconi	Maurizio	Reseacher	Engineering

StudentID	Surname	Name	Program	Year	Supervisor	Code	Surname	Name	Role	Department
276545	Rossi	Maria	bachelor	1	null	FT	Monreale	Anna	Reseacher	Computer Science
276545	Rossi	Maria	bachelor	1	null	CV	Tesconi	Maurizio	Reseacher	Engineering
485745	Neri	Anna	bachelor	2	null	FT	Monreale	Anna	Reseacher	Computer Science
485745	Neri	Anna	bachelor	2	null	CV	Tesconi	Maurizio	Reseacher	Engineering
200768	**Verdi**	**Fabio**	**bachelor**	**3**	**FT**	**FT**	**Monreale**	**Anna**	**Reseacher**	**Computer Science**
200768	Verdi	Fabio	bachelor	3	FT	CV	Tesconi	Maurizio	Reseacher	Engineering
587614	**Rossi**	**Luca**	**master**	**2**	**FT**	**FT**	**Monreale**	**Anna**	**Reseacher**	**Computer Science**
587614	Rossi	Luca	master	2	FT	CV	Tesconi	Maurizio	Reseacher	Engineering
937653	Bruni	Mario	master	1	CV	FT	Monreale	Anna	Reseacher	Computer Science
937653	**Bruni**	**Mario**	**master**	**1**	**CV**	**CV**	**Tesconi**	**Maurizio**	**Reseacher**	**Engineering**

Example of JOIN

STUDENTS

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	2	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

TEACHERS

Code	Surname	Name	Role	Department
FT	Monreale	Anna	Reseacher	Computer Science
CV	Tesconi	Maurizio	Reseacher	Engineering

StudentID	Surname	Name	Program	Year	Supervisor	Code	Surname	Name	Role	Department
200768	Verdi	Fabio	bachelor	3	FT	FT	Monreale	Anna	Reseacher	Computer Science
587614	Rossi	Luca	master	2	FT	FT	Monreale	Anna	Reseacher	Computer Science
937653	Bruni	Mario	master	1	CV	CV	Tesconi	Maurizio	Reseacher	Engineering

FROM

- **Strategy a**

```
FROM R,S
WHERE S.A=R.B
```

- **Strategy b**

```
FROM S JOIN R ON S.A=R.B
```

Example: JOIN

- List students with a mark greater than 26 in the exam for databases

```
SELECT S.StudentID, S.surname, S.name
FROM Students S JOIN Exams E ON
S.StudentID=E.student JOIN Courses C ON C.code =
E.course
WHERE E.mark > 26 AND C.title = 'databases'
```

Set Operators

- Binary Operators

- `Union`: R U S
- `Intersection`: R ∩ S
- `EXCEPT/MINUS`: R – S

Set Operators

- Tables R and S must have the same number of attributes
- **Positional Association**
  - The list of attributes in the SELECT must have the same type
- **Result Schema**
  - Attribute names of the first table
- **Attention**
  - Elimination of duplicates
  - Otherwise use: UNION ALL, INTERSECT ALL, EXCEPT ALL

Union

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated ∪ specialist

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

```
SELECT *
FROM Graduated

UNION

SELECT *
FROM Specialist
```

Union

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated ∪ specialist

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45
9297	Neri	33

```
SELECT StudentID, Name, Surname
FROM Graduated

UNION

SELECT StudentID, Surname,Name
FROM Specialist
```

Intersection

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated ∩ Specialist

StudentID	Name	Age
7432	Neri	54
9824	Verdi	45

```
SELECT *
FROM Graduated

INTERSECT

SELECT *
FROM Specialist
```

Intersection

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	trenta
7432	Neri	venti
9824	Verdi	quaranta

```
SELECT *
FROM Graduated

INTERSECT

SELECT *
FROM Specialist
```

```
SELECT StudentID, name
FROM Graduated

INTERSECT

SELECT StudentID, name
FROM Specialist
```

Intersection: IN

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated ∩ Specialist

StudentID	Name	Age
7432	Neri	54
9824	Verdi	45

```
SELECT *
FROM Graduated
WHERE StudentID IN (

SELECT StudentID
FROM Specialist
)
```

Difference

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated– Specialist

StudentID	Name	Age
7274	Rossi	42

```
SELECT *
FROM Graduated

EXCEPT

SELECT *
FROM Specialist
```

Difference: NOT IN

Graduated

StudentID	Name	Age
7274	Rossi	42
7432	Neri	54
9824	Verdi	45

Specialist

StudentID	Name	Age
9297	Neri	33
7432	Neri	54
9824	Verdi	45

Graduated– Specialist

StudentID	Name	Age
7274	Rossi	42

```
SELECT *
FROM Graduated
WHERE StudentID NOT IN(
        SELECT StudentID
        FROM Specialist
)
```

GROUP BY

Anna Monreale

Quering a DB

- **Core of the SELECT Instruction**
  - SELECT: projection, rename, distinct
  - FROM: cartesian product or join, alias
  - [WHERE]: selection
- **Other components**
  - [ORDER BY]
  - [GROUP BY]
  - [HAVING]

Aggregate Queries

- Average mark and total number of exams

```
SELECT AVG(mark), COUNT(*)
FROM Exams
```

- Average mark and total number of exams of student with StudentID = 1000

```
SELECT AVG(mark), COUNT(*)
FROM Exams
WHERE student = 1000
```

Aggregate Function

- Expressions computing functions starting from a set of tuples
  - Count the number of tuples: `COUNT`
  - Minimum value of an attribute: `MIN(attribute)`
  - Maximum value of an attribute : `MAX(attribute)`
  - Avarage value of an attribute: `AVG(attribute)`
  - Sum of values of an attribute: `SUM(attribute)`

- Syntax:
  - Function([DISTINCT] *)
  - Function([DISTINCT] Attribute)

What about returning avarage mark and number of exams student by student?

```
SELECT student, AVG(mark), COUNT(*)
FROM Exams
```

ILLEGAL!

Which student?

What we need …

EXAMS

Student	Mark	Laud	Course
276545	28	0	01
276545	27	0	04
937653	25	0	01
200768	30	1	04
587614	28	0	03

1. Create different groups
2. For each group compute the function

GROUP BY

- **GROUP BY**
  - Grouping operator
- **Syntax**
  - `GROUP BY <Attribute List>`
- **Semantics**
  - Create groups of tuples
  - Each group has the same value for the grouping attributes

Example GROUP BY

- Grouping students by per Program

```
SELECT Program
FROM Students
GROUP BY Program
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	1	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

Example GROUP BY

- Grouping Students per Program and Year

```
SELECT Program, Year, count(*)
FROM Students
GROUP BY Program, Year
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	1	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

Example GROUP BY

- Grouping Students per StudentID

```
SELECT StudentID
FROM Students
GROUP BY StudentID
```

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	1	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

…. so our query is ….

EXAMS

Student	Mark	Laud	Course
276545	28	0	01
276545	27	0	04
937653	25	0	01
200768	30	1	04
587614	28	0	03

1. Create different groups
2. For each group compute the function

Student	AVG_mark	NExams
276545	27.5	2
200768	30	1
587614	28	1
937653	25	1

```
SELECT Student, AVG(Mark) AS AVG_Mark, COUNT(*)AS NExams
FROM Exams
GROUP BY Student
```

Queries with GROUP BY

- **Grouping Attributes**
  - GROUP BY
- **Projection of grouping attributes**
  - SELECT
- **Aggregate Functions applied to the group**
  - SELECT
- **Conditions on the groups involving aggregate functions**
  - HAVING

Example

- Average mark per course

```
SELECT Course, AVG(Mark) AS AVG_MARK
FROM Exams
GROUP BY Course
```

Course	AVG_Mark
01	26.5
04	28.5
03	28

EXAMS

Student	Mark	Laud	Course
276545	28	0	01
276545	27	0	04
937653	25	0	01
200768	30	1	04
587614	28	0	03

Example COUNT

- Numero di Students per Program

```
SELECT Program, COUNT(*)
FROM Students
GROUP BY Program;
```

Program	Count(*)
bachelor	3
master	2

```
SELECT Program, COUNT(*) AS NumStudents
FROM Students
GROUP BY Program;
```

Program	NumStudents
bachelor	3
master	2

Sematics:

- Evaluation of FROM
- Creation of groups with GROUP BY
- Evaluation of SELECT **for each group**
- Each group contributes with ONLY ONE tuple in the result

GROUP BY & Constraints in SELECT

- If there is `GROUP BY`, only grouping attributes may appear in `SELECT`

```
SELECT Program, COUNT(*)
FROM Students
GROUP BY Program;
```

CORRECT!

```
SELECT COUNT(*)
FROM Students
GROUP BY Program;
```

CORRECT!

```
SELECT Year, COUNT(*)
FROM Students
GROUP BY Program;
```

WRONG!

Example COUNT

- Distribution per Year of bachelor students

```
SELECT Year, COUNT(*) AS NumStud
FROM Students
WHERE Program='bachelor'
GROUP BY Year;
```

Year	NumStud
1	2
3	1

StudentID	Surname	Name	Program	Year	Supervisor
276545	Rossi	Maria	bachelor	1	null
485745	Neri	Anna	bachelor	1	null
200768	Verdi	Fabio	bachelor	3	FT
587614	Rossi	Luca	master	2	FT
937653	Bruni	Mario	master	1	CV

HAVING

- Expresses conditions on groups
- It is not possible to use `WHERE`
- **Example:**
  - **Avearge Mark for each course with at least 2 exams**

EXAMS

Student	Mark	Laud	Course
276545	28	0	01
276545	27	0	04
937653	25	0	01
200768	30	1	04
587614	28	0	03

```
SELECT Course, AVG(Mark) AS AVG_Mark
FROM Exams
GROUP BY Course
HAVING COUNT(mark)>1
```

Course	AVG_Mark
01	26.5
04	28.5

QUERY

```
SELECT [DISTINCT] <result>
FROM <join or cartesian product>
[WHERE <condition on tuples>]
[GROUP BY <grouping attributes>
[HAVING <conditions on groups>]
[ORDER BY <ordering attributes>]
```